I/O Techniques

Week 8 — How a CPU Talks to the Outside World

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

August 14, 2026

Where We Left Off (Week 7)

Last week: how the CPU controls *itself*

- Hardwired control (Week 6) and microprogrammed control (Week 7) both answered the same question: how does the CPU generate its own control-step sequence, cycle by cycle?
- Both designs looked *inward* — registers, ALU, and an internal bus.

The gap Week 7 left open

Keyboards, disks, and printers are far slower than the CPU. How does the CPU get data from them — and what does the *waiting* cost?

Roadmap: Four Questions, One Thread

1. **Program-controlled I/O (polling)** — the CPU keeps asking "are you ready?" until the device says yes.
2. **Interrupt-driven I/O** — the device asks the CPU instead.
3. **Synchronous vs. asynchronous buses** — what "ready" looks like on the wires between CPU and device.
4. **DMA and I/O processors** — removing the CPU from the data path entirely.

Running thread for today

One keyboard-input / display-output pair (`KIN`/`DOUT`) follows us through polling and interrupts; a 4096-byte disk block follows us through bus timing and into DMA/IOP.

Before any technique: what is the problem they all solve?

Socratic Check: The Speed Mismatch

Question

A CPU runs one instruction about every nanosecond (10^9 instructions per second). A keyboard produces one keystroke about every 200 milliseconds (about 5 keystrokes per second). If the CPU must read every keystroke, what should it do during the 200 ms between two keystrokes?

- The CPU is roughly **200 million times** faster than the typist — the two operate on completely different time scales.
- Waiting wastes the CPU, so I/O design is really the question: *who does the waiting*, and *what does the waiting cost*?
- Today's four techniques are four answers to that one question.

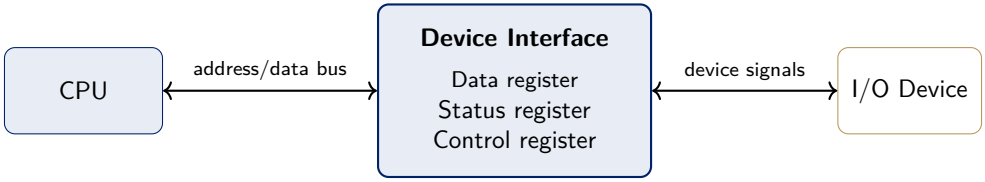
The Simplest Way In: Memory-Mapped I/O

Memory-mapped I/O

A device's **data**, **status**, and **control** registers are given addresses in the *normal* memory space. The CPU reads and writes a device exactly like it reads and writes memory — no special I/O instructions needed [1].

- The alternative — **isolated I/O** — needs separate I/O instructions and a separate address space (the classic IBM approach, Mano Ch. 11). Memory-mapped I/O is simpler for this course.

The I/O Device Interface: Three Registers



What this diagram shows: the CPU never talks to the device directly. It talks to three registers inside the device's interface, over the ordinary bus:

- **Data register** — the byte actually being transferred (e.g. `KBD_DATA` for the keyboard, `DISP_DATA` for the display).
- **Status register** — flags such as `KIN` (a keystroke is waiting) or `DOUT` (the display is ready for the next character).
- **Control register** — settings such as interrupt-enable bits.
- The interface also performs address decoding, timing, and format conversion — the "six functions" of an interface circuit, Hamacher Ch. 7 §7.4 [1, p. 238].

Memory-Mapped vs. Isolated I/O: The Trade-off

	Memory-mapped I/O	**Isolated I/O**
Address space	Devices share the memory space	Separate I/O space
Instructions	Ordinary `Load`/`Store`	Dedicated I/O instructions
Simplicity	Simpler for the programmer	Extra hardware/instructions
Typical use	Most modern processors	Classic IBM mainframes

We use memory-mapped I/O for the rest of the deck — it is what the Hamacher examples assume [1]. The isolated form is the standard alternative (Mano Ch. 11).

Technique 1: the CPU does all the asking — polling.

Definition: Program-Controlled I/O (Polling)

Polling

In **program-controlled I/O** (also called **polling**), the CPU runs a loop that repeatedly reads the device's *status register* and checks a ready flag. When the flag says the device is ready, the CPU stops the loop and performs the data transfer itself.

- The device never initiates anything — it only answers questions.
- Two new status flags, one per device direction: `KIN` (a keyboard keystroke is ready in `KBD_DATA`) and `DOUT` (the display is ready for the next character in `DISP_DATA`).

Worked Example: Two Polling Loops

Keyboard input

Read one keystroke into register `R`.

READWAIT loop

```
READWAIT: Test KIN
          Branch=0 READWAIT
          MoveByte KBD_DATA, R
```

Display output

Send register `R` to the display.

WRITEWAIT loop

```
WRITEWAIT: Test DOUT
           Branch=0 WRITEWAIT
           MoveByte R, DISP_DATA
```

How to read these loops:

- `Test KIN` reads the status flag; `Branch=0` jumps back while the flag is still `0` (not ready).
- Only when the flag turns `1` does the loop fall through to the single transfer instruction.
- Each pass runs the same two-test instructions again — the CPU is spinning, doing no useful work between keystrokes [1].

Worked Example: The Cost of Polling, in Numbers

The setup

A 2 GHz CPU (one instruction every 0.5 ns) polls a keyboard that produces 5 keystrokes per second. Each poll takes one 2-instruction pass (about 1 ns). Between two keystrokes, how many polls run, and how much CPU time do they waste?

- Gap between keystrokes: $1/5 = 0.2$ s $= 2 \times 10^8$ ns.
- Polls per gap: 2×10^8 ns $\div$ 1 ns/poll $= 2 \times 10^8$ **polls wasted** per keystroke.
- Fraction of CPU consumed by polling: 2×10^8 ns $\div$ 2×10^8 ns $= 1.0$ — the CPU spends **100%** of its time waiting on this one keyboard.

Moral: polling is trivially simple but hands the CPU over to the slowest device in the system.

Polling: Trade-offs

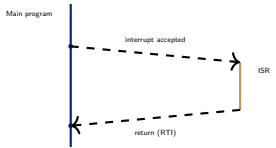
Advantage	Disadvantage
No extra hardware needed	Wastes CPU time while waiting
Trivial to implement	CPU cannot do other work during the wait
Works for any device	Cost grows with device slowness

When polling is (and is not) a good fit

Good fit: one slow device and a CPU with nothing else to do. **Bad fit:** a fast device, or several devices — the CPU spends its life asking questions instead of computing.

Technique 2: what if the device asked the CPU, instead of the CPU asking the device?

Definition: Interrupt and Interrupt-Service Routine



Interrupt

An **interrupt** suspends the CPU's current program and forces a jump to a fixed handler — the **interrupt-service routine (ISR)** — which services the device, then returns control exactly where it left off.

- Unlike polling, the device now signals *once*, unprompted — the direction of initiative reverses.
- **Interrupt latency**: the gap between the device's request and the CPU's jump — time during which the device waits.
- **Shadow registers**: registers where the CPU saves state so the ISR can use them freely [1].

Enabling and Disabling Interrupts: Two Switches

A request reaches the CPU only if *both* switches are on

- `IE` — one interrupt-enable bit inside the processor status register (`PS`). When `IE`$= 0$, the CPU ignores *every* interrupt request, from any device.
- A *per-device* interrupt-enable bit in that device's control register — `KIRQ` for the keyboard, `DIRQ` for the display — decides whether *this* device may request an interrupt at all.

- Example: `KIRQ`$= 1$ and `IE`$= 1$ means a keystroke will interrupt; `IE`$= 0$ means it waits.

Worked Example: A 5-Step Interrupt Scenario

The story: a keystroke arrives while the CPU is busy running a program. Here is exactly what happens, step by step.

1. The device sets its status flag (`KIN`$= 1$) and, if its enable bit `KIRQ` is on, raises an interrupt-request line.
2. The CPU finishes its *current instruction* (never interrupting mid-instruction), then checks the master switch `PS.IE`.
3. If `IE`$= 1$: the CPU saves `PC` (and any registers the ISR will overwrite) into shadow registers, disables further interrupts, and loads `PC` with the ISR's starting address.
4. The ISR services the device — here, it reads `KBD_DATA` (which clears `KIN`) — then executes a return-from-interrupt instruction.
5. Return-from-interrupt restores the saved `PC` and re-enables interrupts. The interrupted program resumes as if nothing happened [1].

Notice what the CPU did *not* do: it did not poll. It went on computing until the device rang the bell.

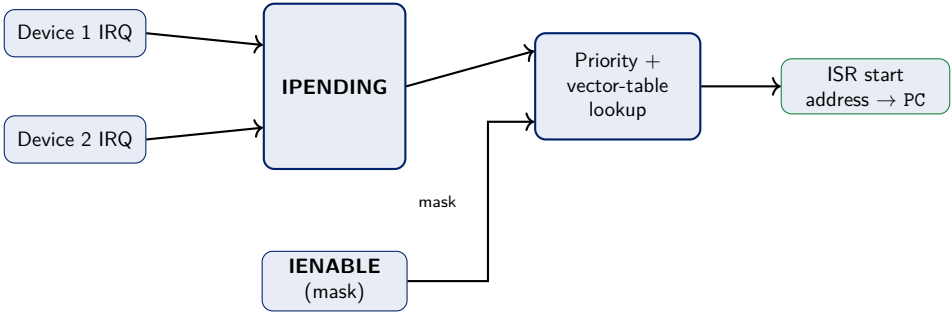
Handling Multiple Devices: Who Interrupted?

Two designs for answering "which device interrupted?"

- **Polling the sources**: on any interrupt, the ISR checks each device's status flag in turn to find the one that rang — simple, but the check order costs time proportional to the number of devices.
- **Vectored interrupts**: each device supplies a small identifying code; hardware uses it to index straight into an **interrupt-vector table** — one ISR starting address per device — and jumps directly there.

- Trade-off: a little extra hardware (the table + lookup) buys **constant-time dispatch**, no matter how many devices [1].
- Quantified: 8 devices cost up to **8 sequential checks** under polling-the-sources, but exactly **1 lookup** under a vector table, regardless of device count.

Interrupt Nesting and Priority: Two Devices, One CPU



What this diagram shows: `IPENDING` latches *which* devices currently want an interrupt; `IENABLE` decides *which of those* are allowed through; the priority encoder picks the highest-priority allowed one; the vector table gives its ISR address. If the lower-priority device's request stays pending while the higher one is being serviced, it waits until interrupts are re-enabled — or, with **nesting**, a higher-priority request can interrupt the ISR itself.

Exceptions: The General Form of an Interrupt

Exception

An **exception** is any event that interrupts the normal flow of control — an I/O interrupt, but also divide-by-zero, an illegal instruction, a page fault, or a timer tick. The interrupt mechanism is the same; only the *source* differs [1, 2].

- Thinking of them together matters because the CPU handles them with the same jump-to-handler machinery: save state, service, restore.
- This is the bridge the next lectures will reuse: exceptions are how the OS gets control back from user programs.

Worked Example: An Exception in Action

The scenario

A running program executes `DIV R1, R2, R3` where `R3` holds `0` — not an I/O event, but the CPU handles it with exactly the same machinery Act 3 already built.

1. The ALU detects the divide-by-zero and raises a fixed internal exception signal — no external device, no interrupt-request line needed.
2. The CPU finishes what it can of the current instruction, then forces a jump; unlike an ordinary interrupt, this jump is **not** gated by `PS.IE` — exceptions cannot be masked.
3. `PC` and status are saved into shadow registers; `PC` loads the divide-by-zero handler's fixed address.
4. The handler runs (report the error, or hand control to the OS) — the same save/service/restore shape as an ISR, just a different table entry.

Same machinery, different trigger: a page fault, an illegal opcode, and a timer tick all follow this identical path, each with its own starting address in step 3 [1, 2].

Socratic Check: Simultaneous Interrupt Requests

Question

Device 1 (higher priority) and Device 2 both request an interrupt on the same cycle. Device 1's ISR is still running when Device 2's request would normally be granted. What happens?

- If **nesting** is allowed and Device 2 outranks whatever priority level Device 1's ISR is running at, Device 2 can interrupt *that ISR* — the same 5-step scenario applies recursively.
- If nesting is disabled (or Device 2 is lower priority), Device 2's request stays latched in `IPENDING` until Device 1's ISR re-enables interrupts.

Polling vs. Interrupts: The Trade-off

	Polling	**Interrupts**
Who initiates	CPU (asks repeatedly)	Device (signals once, when ready)
CPU cost while waiting	Wasted spin cycles	Zero — CPU runs other code
Extra hardware	None	`IE`, per-device enables, vector table
Response latency	Depends on poll rate	Interrupt latency (usually smaller)
Best fit	One slow device, nothing else to do	Multiple/unpredictable devices

Same problem, opposite direction of initiative — interrupts move the waiting cost from the CPU to the hardware that detects readiness.

One level down: what does “the device is ready” look like on the wires?

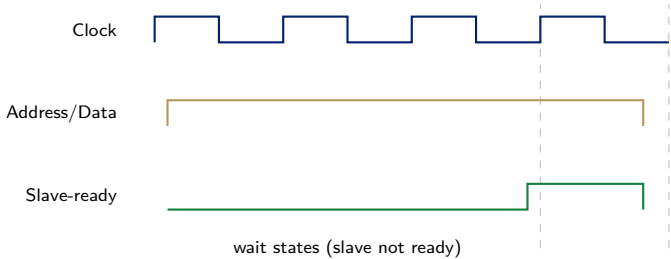
Bus Structure: Master and Slave Roles

One shared bus, many devices

Every I/O interface from Act 1 hangs off a single shared **bus** — address, data, and control lines that every device listens to. A **bus protocol** is the rulebook for how a **master** (the device driving a transfer, usually the CPU) and a **slave** (the device responding) coordinate one data transfer without colliding.

- **This Act's question**: when the master says "take this data" or "give me that data," how does the slave say "not ready yet" or "here it is"?
- Two different answers: **synchronous** (everyone shares a clock) and **asynchronous** (an explicit handshake, no clock) [1].
- Running example for this Act: our 4096-byte disk block's controller is today's slave — every timing diagram below is the CPU (master) and that disk interface negotiating one word of the transfer.

Definition: Synchronous Bus



Synchronous bus

All devices share a common **bus clock**: the master places address/data at a fixed clock edge, and every slave samples that *same* edge, no per-transfer negotiation.

If our disk controller is slower than the clock, it holds `Slave-ready` low for extra clock cycles while the CPU simply waits, still synchronized to the shared clock [1].

Worked Example: How Many Wait States?

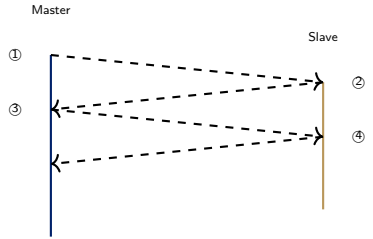
The setup

Our disk controller takes 100 ns to place a word on the bus. The synchronous bus clock runs at 500 MHz (one clock cycle $= 2$ ns). How many wait-state cycles does the master insert for one word?

- Clock cycle time: $1/500\text{MHz} = 2$ ns.
- Cycles needed to cover the disk's 100 ns response: $100 \div 2 = \mathbf{50}$ clock cycles.
- One cycle is the "normal" data cycle, so the master inserts **49 wait-state cycles** — the slave holds `Slave-ready` low for all 49 before the transfer completes.

Compare with Act 2's polling cost (2×10^8 wasted polls for one keystroke): a slow device is expensive under every technique — the question is only *who* pays and *how much*.

Definition: Asynchronous Bus (Full Handshake)



Asynchronous bus

No shared clock. Master and slave exchange explicit **Master-ready**/**Slave-ready** signals, each changing only *in response to* the other — a **full handshake**.

① Master-ready asserted
② Slave-ready (data valid)
③ Master-ready withdrawn (latched)
④ Slave-ready withdrawn

Each step waits for the previous one, so **bus skew** (wires arriving at slightly different times) never causes a false sample, unlike a synchronous bus's single fixed clock edge [1].

Synchronous vs. Asynchronous: The Trade-off

	Synchronous	**Asynchronous**
Timing reference	Shared bus clock	Explicit handshake, no clock
Slow-device handling	Extra wait-state clock cycles	Handshake simply takes longer
Skew sensitivity	Sensitive — must beat clock period	Immune — self-timed
Per-transfer overhead	Low (one clock edge)	Higher (two round trips)
Best fit	Devices of similar, known speed	Mixed-speed / unpredictable devices

Same coordination problem, opposite mechanism — a shared clock everyone trusts, or an explicit conversation that trusts nothing.

Socratic Check: Why Not Always Use Asynchronous?

Question

Asynchronous handshaking tolerates skew and adapts automatically to any device speed. Why do most high-speed processor buses use synchronous protocols instead?

- Every asynchronous transfer pays *two full round-trip delays* (steps 1–2 and 3–4) — for consistently fast devices this is pure overhead a synchronous bus does not pay.
- Synchronous buses only lose when a device is unusually slow (extra wait states) or the clock must shrink to accommodate skew — exactly the trade-off the comparison table names.

Interrupts still cost one instruction per word transferred — what if the CPU left the loop entirely?

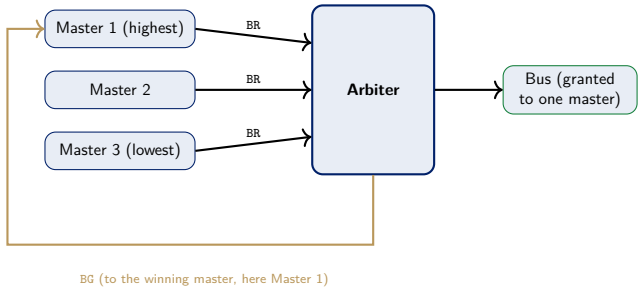
The Cost Interrupts Still Pay

Question

A disk transfer moves a 4096-byte block, one 4-byte word at a time. With interrupt-driven I/O, the ISR executes on *every word*: read status, read/write data, update a memory pointer, decrement a counter, return. What does this cost for one block?

- Words per block: $4096 \div 4 = \mathbf{1024}$ words.
- That means **1024 ISR invocations** for a single block transfer — one interrupt per word.
- Interrupts removed the CPU's *waiting*, but not its *involvement in moving every byte*. The next step removes the CPU from the data path itself.

Bus Arbitration: A Third Role for a Device



Any device connected via BR/BG can become bus **master**, not just the CPU — the arbiter grants the bus to exactly one requester, by priority, each time [1]. Being bus master is how a device can talk to memory *directly* — the key idea the next slide builds on.

From Arbitration to Direct Memory Access

The functional idea, stated plainly

Once a device can become bus master, it can transfer data *directly to or from memory* — no CPU instruction touches the data at all. The CPU is interrupted *once*, when the whole block finishes, not once per word.

This edition of Hamacher's text (Ch. 7, arbitration) describes exactly this mechanism but never names it "DMA" — the term and its standard treatment below follow Stallings Ch. 8 [3], per the syllabus and `.claude/rules/textbook-grounding.md`.

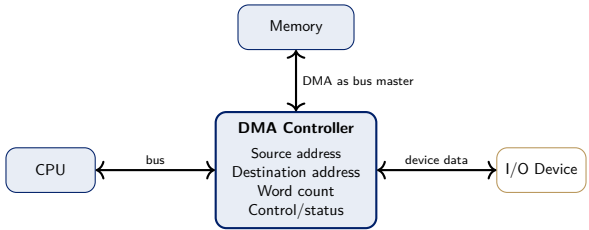
Definition: Direct Memory Access (DMA)

DMA (general/standard treatment — Stallings Ch. 8)

A **DMA controller** is given a source/destination address and a word count by the CPU, then becomes bus master (via arbitration) and transfers the entire block directly to/from memory, interrupting the CPU only when the *block* — not each word — is complete [3].

- Pro: CPU cost drops from 1024 ISRs to "one setup, one completion interrupt" — a constant cost regardless of block size.
- Con: the DMA controller and CPU now compete for the same memory bus (**cycle stealing**); extra hardware required.

The DMA Controller: What Is Inside



What each register does:

- **Source / destination address** — where the data comes from and goes to (memory address and device, or vice versa).
- **Word count** — how many words remain to transfer; decremented after every word.
- **Control/status** — transfer direction, mode, and the "block done" flag that interrupts the CPU.
- The CPU writes all four registers once at setup; the DMA controller owns them from then on.

How a DMA Transfer Runs: Step by Step

The story: copy the 4096-byte block from the disk to memory. The CPU appears twice — at setup and at completion — and never touches the data itself.

1. **Setup:** the CPU writes the DMA controller's registers — source address (disk), destination address (memory), word count (1024), and direction (read) [3].
2. **Request:** the DMA controller asks for the bus by asserting `BR` (bus request).
3. **Grant:** the arbiter answers with `BG` (bus grant); the DMA controller becomes bus master [1].
4. **Transfer one word:** the DMA controller reads one word from the device and writes it to memory (or the reverse), using one or two bus cycles.
5. **Decrement:** it subtracts 1 from the word count and, if the count is still > 0, returns to step 2 for the next word.
6. **Done:** when the count reaches 0, the DMA controller drops `BR` and raises a *completion interrupt* — the CPU's only involvement after setup.

Compare with polling/interrupts: the CPU now appears exactly twice, at the start and the end.

DMA Transfer Modes: Three Ways to Use the Bus

How long does the DMA controller keep the bus?

- **Burst (block) mode**: holds the bus for the *entire* block — all 1024 words in one continuous burst, then releases and interrupts the CPU.
- **Cycle-stealing (single) mode**: grabs the bus for *one word*, releases it, re-requests — the CPU can use the bus between DMA words.
- **Transparent (hidden) mode**: transfers *only during CPU cycles that do not use the bus at all* — the CPU never even notices the DMA happening.

	Burst	**Cycle stealing**	**Transparent**
Bus held	Whole block	One word	Only idle CPU cycles
Transfer speed	Fastest	Medium	Slowest
Effect on CPU	Starved for the block	Brief pauses per word	None

In short:

burst = "whole job now, do not bother me"; cycle stealing = "one word, then back off"; transparent = "only when you're not looking."

Worked Example: How Much CPU Does DMA Save?

The comparison

Same 4096-byte block as the interrupt example. Interrupt-driven I/O: 1024 ISRs. DMA: one setup + one completion interrupt. Suppose one ISR takes 200 CPU instructions; how many instructions does each technique spend?

- Interrupts: $1024 \times 200 = \mathbf{204{,}800}$ instructions.
- DMA: one setup + one completion ISR ≈ 400 instructions — roughly $\mathbf{500\times}$ **less** CPU work.
- The CPU that used to spend its whole time servicing the disk can now run the user's program instead.

Definition: I/O Processor (IOP)

IOP (general/standard treatment — Stallings Ch. 8)

An **I/O processor** generalizes DMA one step further: a dedicated processor, given a whole *channel program* (a short program describing several transfers, formats, and simple decisions), executes it independently — interrupting the main CPU only when the entire program completes, not per block.

- The difference is *how much decision-making is delegated* away from the CPU, not just how the data moves.

Worked Example: A Channel Program for IOP

The scenario

A print job needs three things done in order: read a 4096-byte record from disk, convert it to the printer's character format, then send it to the printer — and interrupt the CPU only when all three finish.

1. **Setup:** the CPU writes a short *channel program* into memory — three steps: `READ disk` → `buffer`, `CONVERT buffer` → `buffer2`, `WRITE buffer2` → `printer` — then hands the IOP its starting address.
2. **Execution:** the IOP runs all three steps itself, becoming bus master as needed (exactly like DMA), with *no* CPU involvement between steps.
3. **Completion:** only after `WRITE` finishes does the IOP raise one interrupt — the CPU never sees the intermediate `READ`/`CONVERT` steps at all.

Compare with DMA: DMA's "program" is always the same one step (copy this block); an IOP's channel program can chain several different operations — exactly the extra delegation Stallings Ch. 8 describes [3].

Four I/O Techniques, Head to Head

	Polling	**Interrupts**	**DMA**	**IOP**
CPU cost / word	Every poll	Every word (ISR)	None (setup only)	None
CPU cost / block	—	Thousands of ISRs	One interrupt	One interrupt
Extra hardware	None	Vector table, `IE`	DMA controller	Dedicated processor
Best fit	One slow device	Multiple/sporadic devices	Bulk block transfer	Complex I/O subsystems

Same core question, four answers: who watches the device, and how much of the CPU does watching cost?

Socratic Check: Choosing a Technique

Question

A system reads one temperature sensor value per second (low rate, latency-tolerant) and separately copies a 10 MB file from disk to memory (high rate, throughput-critical). Which technique fits each?

- Temperature sensor: **interrupts** — rate is low, so ISR overhead per event is negligible, and the CPU is free the rest of the time.
- Disk copy: **DMA** — per-word CPU involvement at that data rate would dominate CPU time; DMA reduces it to one setup, one completion interrupt.
- Rule of thumb: match the technique to the *rate* and *size* of the transfer, not to habit.

Four techniques, one trade-off. What is the pattern underneath?

The Four Threads Meet

Every design choice moves the waiting cost somewhere else

- **Polling vs. interrupts (programmer's view):** polling wastes CPU cycles asking (recall: 2×10^8 wasted polls for one keystroke); interrupts let the device ask instead, at the cost of vector-table/priority hardware.
- **Synchronous vs. asynchronous (bus hardware):** a shared clock is cheap per transfer but skew-sensitive (recall: 49 wait states for one slow disk word); a handshake is skew-immune but pays two round trips every time.
- **Interrupts vs. DMA/IOP (who moves the data):** interrupts still touch every word (recall: 204,800 instructions for one disk block); DMA and IOP remove the CPU from the data path, trading extra hardware for near-zero per-word cost (≈ 400 instructions for the same block).

None of these is free — exactly the pattern Weeks 5–7 established for buses and control units, now applied to the world outside the chip.

Bridge to Week 9

From moving data to storing it efficiently

Weeks 1–8 built a CPU that computes, controls itself, and now moves data to and from slow devices. Week 9 asks: once data is in memory, how do we make *memory itself* fast enough to keep up with the CPU?

- DRAM is far slower than the CPU — the same speed-mismatch problem this week solved for devices reappears for memory itself.
- Cache memory, mapping functions, and replacement policies are Week 9's answer.

Summary

- **Memory-mapped I/O**: device registers (data/status/control) live in the normal address space; **polling** repeatedly tests a status flag (`KIN`, `DOUT`).
- **Interrupts**: the device signals readiness; `PS.IE` and per-device enables (`KIRQ`/`DIRQ`) gate delivery; **vectored interrupts** use `IPENDING`/`IENABLE` plus an interrupt-vector table for constant-time dispatch.
- **Synchronous bus**: shared clock, wait states for slow slaves. **Asynchronous bus**: `Master-ready`/`Slave-ready` full handshake, skew-immune but higher per-transfer overhead.
- **Bus arbitration** (`BR`/`BG`) lets a device become bus master — the functional basis for **DMA** (block transfer, one completion interrupt) and **IOP** (whole channel programs, general/standard treatment per Stallings Ch. 8).
- **Four techniques, one trade-off axis**: how much of the CPU does watching and moving the data cost, and how much hardware buys that cost down.
- Every number today was one bill for the same speed mismatch — the techniques differ only in who pays it.

References

V. Carl Hamacher, Zvonko G. Vranesic, Safwat G. Zaky, and Naraig Manjikian.
Computer Organization and Embedded Systems.
McGraw-Hill, 6th edition, 2012.
ISBN 978-0-07-338065-0. Bib key retained as Hamacher2002_computer_organization for citation-key stability across existing lectures; the file indexed under master_supporting_docs/CS401/supporting_books/Hamacher2002/ and page-cited by Week 8 is this 6th edition, not the 5th ed. (2002) the key name suggests – see that folder's index.md Edition notice, 2026-08-11.

David A. Patterson and John L. Hennessy.
Computer Organization and Design: The Hardware/Software Interface.
Morgan Kaufmann, arm edition (5th) edition, 2017.

William Stallings.
Computer Organization and Architecture: Designing for Performance.
Pearson, 10th edition, 2015.